
PART IV — FROM RAG TO AGENTIC RAG

Chapter 17

Tool Use and Function Calling

“A useful agent does not merely reason about the world; it acts through contracts.”

Chapter 17 — Tool Use and Function Calling

The loop in Chapter 16 can choose an action, but choice becomes useful only when the application can execute it safely. Tool-calling is the bridge between model output and program behavior. The model emits a structured request; the orchestrator validates it; an ordinary function performs the operation; and the result returns to the model under the identifier of the original call.

Memora’s history shows why this boundary deserves its own chapter. Tool schemas enabled iterative retrieval, but early designs exposed procedural choices that belonged in configuration, allowed incompatible actions in the same phase, and encountered provider-specific failures such as `‘tool_use_failed’` and malformed `‘function.arguments’`. The fixes were architectural: minimize the schema, inject server context privately, validate before execution, preserve call/result pairing, and move quality judging out of the model-visible tool set. By the end, you will be able to design a tool that is useful, narrow, testable, and safe.

17.1 What a tool is from the LLM’s perspective

Definition — Tool

An application-provided operation described to the model by a name, purpose, and argument schema. The model may request the operation, but the application retains authority to validate, execute, reject, or defer it.

From the model’s perspective, a tool is not a Python function object. It is text plus structure: a name such as `‘retrieve_documents’`, a description of when to use it, and a JSON-style schema describing permitted arguments. The model’s response contains a call record rather than the tool’s actual result. Only the host program can cross from that request into code execution.

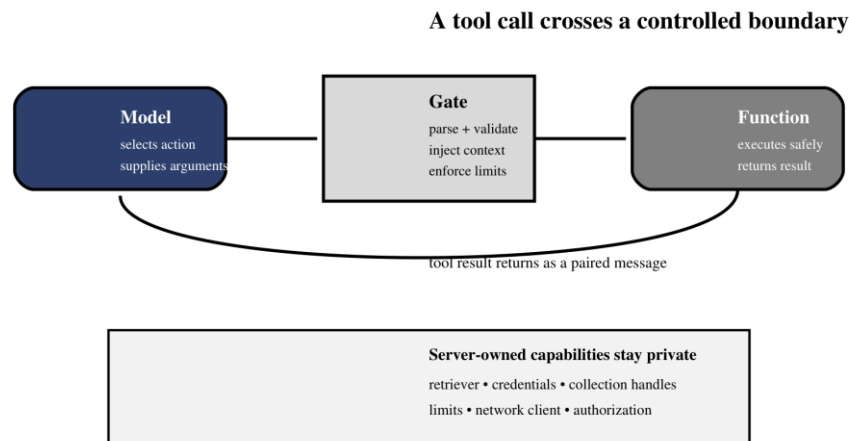


Figure 17.1 — The orchestrator mediates every transition from model intent to executable code.

As Figure 17.1 emphasizes, neither a plausible name nor schema-shaped arguments grant authority. The request crosses the boundary only after application policy has approved the operation and supplied trusted context.

This separation matters because the model is probabilistic and the function is operational. The model can misspell a field, invent an argument, request a forbidden path, or call the right tool at the wrong phase. The orchestrator converts a suggestion into a controlled action. It also supplies dependencies the model

should never see: retriever objects, database clients, credentials, collection handles, and authorization rules.

Analogy — A restaurant order ticket

A diner chooses from the menu and states preferences. They do not enter the kitchen, operate the stove, or choose the supplier account. The waiter checks the order, the kitchen performs an approved operation, and the completed dish returns with the ticket that identifies who requested it.

17.2 Writing tool schemas in JSON Schema style

A strong schema makes valid requests easy and invalid requests obvious. Keep the name action-oriented, write a description that states when the tool should and should not be used, define the smallest argument set, forbid extra properties, and constrain strings or enumerations where possible. Descriptions should explain semantics; the schema should enforce shape.

```
RETRIEVE_SCHEMA = {
  "type": "function",
  "function": {
    "name": "retrieve_documents",
    "description": (
      "Search approved document and learned-QA sources. "
      "Use a focused query that names the missing concept."
    ),
    "parameters": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "minLength": 3,
          "maxLength": 500,
        }
      },
      "required": ["query"],
      "additionalProperties": False,
    },
  },
}
```

Schema choice	Why it matters	Weak alternative
One focused verb	Reduces tool-selection ambiguity	<code>`do_rag_stuff`</code>
Behavioral description	Teaches appropriate use	Repeating only the function name
Minimal arguments	Shrinks the model's error surface	Exposing every internal option
Required fields	Rejects incomplete calls early	Guessing missing values
No extra properties	Blocks invented knobs	Silently accepting unknown fields
Length / enum constraints	Bounds payloads and choices	Unrestricted free-form objects

Common pitfall — Treating the description as enforcement

A sentence such as “call this exactly once” may guide the model, but it cannot guarantee behavior. Enforce phase, count, and ordering in code. Natural-language instructions are hints; program checks are contracts.

17.2.1 Worked example: from model request to tool result

Suppose the user asks why an acronym has two meanings. The model decides that its current evidence is insufficient and emits a call rather than prose. The assistant message carries a generated call ID, the registered tool name, and arguments that satisfy the public schema. Nothing has executed yet. The orchestrator now owns the transition from probabilistic intent to deterministic work.

```
# Assistant message proposed by the model
{
  "role": "assistant",
  "tool_calls": [{
    "id": "call_7f2",
    "function": {
      "name": "retrieve_documents",
      "arguments": "{\"query\":\"ASD acronym clinical software\"}"
    }
  }]
}

# Result appended by the orchestrator after validation and execution
{
  "role": "tool",
  "tool_call_id": "call_7f2",
  "content": "DOCUMENTS: ...\\nLEARNED QA: ..."
```

```
}
```

The orchestrator parses the argument string, rejects unknown fields, checks that retrieval is allowed in the current phase, injects the authenticated tenant and configured depth limits, executes the registered function, bounds the returned text, and appends a tool message with the same `call\_7f2` identifier. The next model invocation can now reason over the result. If validation fails, the application still appends a paired rejection record; it never silently executes a guessed correction.

Analogy — Airport baggage handling

A baggage tag does not move a suitcase by itself. It provides a checked identifier that every conveyor and handler uses until the bag reaches the correct flight. A tool-call ID plays the same accounting role across model request, execution, and returned result.

17.3 Hiding server-side parameters with context injection

A tool often needs more information than the model should control. Retrieval requires a retriever instance, configured collection handles, score thresholds, and result limits. These are server-side context. Bind them when the tool is created or inject them through the orchestrator; do not publish them as model-editable arguments.

```
def make_retrieve_tool(retriever, *, top_k, top_l):
    # retriever and depth limits are trusted server context
    def retrieve_documents(query: str) -> str:
        safe_query = validate_query(query)
        result = retriever.retrieve_separate(
            safe_query, top_k=top_k, top_l=top_l
        )
        return serialize_bounded_result(result)

    return retrieve_documents
```

Memora originally exposed retrieval depth to the LLM. Dry runs showed inconsistent values—sometimes too small for coverage, sometimes large enough to inflate context dramatically. The project removed `top\_k` from the schema and introduced separate `RETRIEVAL\_TOP\_K` and `RETRIEVAL\_TOP\_L` configuration values for documents and learned Q&A. The model still chooses the semantic query, while operations owns capacity.

- Inject database clients, credentials, and file-system roots outside the schema.
- Resolve user identity and permissions from the authenticated request, never from a model argument.
- Keep timeout, retry, and result-size limits in configuration.
- Pass only serializable, bounded results back into model context.

17.4 Why exposing context can cause `tool\_use\_failed` errors

Providers validate tool requests against the declared schema and their own protocol rules. When a schema exposes complex objects, optional internal fields, or contradictory instructions, the model is more likely to produce a call the provider rejects. Groq surfaced such failures as HTTP 400 `tool\_use\_failed`; local TGI

testing also returned `function.arguments` as a dictionary even though the OpenAI-compatible protocol expected a JSON-encoded string.

The robust response is layered. First, simplify the schema. Second, normalize provider responses before framework parsing where necessary. Third, classify errors so transient provider failures can retry while genuine bad requests fail quickly. Fourth, keep a graceful path that preserves the run's prior evidence rather than discarding the entire interaction.

```
def normalize_arguments(raw):
    if isinstance(raw, dict):
        return raw
    if isinstance(raw, str):
        return json.loads(raw)
    raise ToolProtocolError("arguments must be object or JSON string")
```

Common pitfall — Retrying every bad request

A malformed schema or forbidden argument will not improve after exponential backoff. Retry connection, timeout, rate-limit, and selected server failures. Fail fast—or ask the model for a corrected call—when validation proves the request itself is invalid.

17.5 Building `tools.py`: retrieval and quality checking

Memora's `tools.py` captures an important evolution. The agent-callable surface contains retrieval and context-compression operations. `retrieve_documents(query)` queries document and learned-QA collections separately, labels their outputs, and returns a bounded string. `compress_context()` consumes state accumulated by earlier retrievals. The quality judge exists in the same module family but is deliberately returned as a direct Python callable, not shown to the agent as a tool.

Operation	Who invokes it	Reason
<code>'retrieve_documents'</code>	Model through tool call	Query wording benefits from semantic choice
<code>'compress_context'</code>	Model in the earlier loop; later enforced by phase	Transforms accumulated evidence after retrieval
<code>'check_answer_quality'</code>	Orchestrator directly	A mandatory gate should not depend on the model choosing to call it

This is a reusable design test: if skipping an operation would violate correctness, consider making it an orchestrator step rather than an optional model tool. Models are good at deciding what evidence to seek. They are less reliable as the sole guardians of rules that must always execute.

17.5.1 Registry and dispatch architecture

Keep the schema registry separate from the executable registry. The first is safe to serialize into a model request; the second contains trusted callables and private dependencies. At dispatch time, look up by exact name, validate with the schema associated with that name, apply phase and authorization policy, and then

call the implementation. This prevents a model-produced string from becoming reflective access to arbitrary Python functions.

```
PUBLIC_SCHEMAS = {
    "retrieve_documents": RetrieveArgs,
    "compress_context": CompressArgs,
}

EXECUTORS = {
    "retrieve_documents": retrieve_with_private_context,
    "compress_context": compress_current_state,
}

def dispatch(call, run):
    if call.name not in EXECUTORS:
        raise ToolRejected("unknown tool")
    enforce_phase(call.name, run.phase)
    args = PUBLIC_SCHEMAS[call.name].model_validate(call.arguments)
    return EXECUTORS[call.name](args, run=run)
```

Failure	Retry?	Orchestrator response
Timeout or selected 5xx	Usually, within budget	Back off, preserve trace, then retry
Provider 429	After advertised delay	Respect shared rate gate and deadline
Malformed JSON arguments	Not blindly	Return a paired validation error or re-prompt
Unknown tool or forbidden phase	No	Reject deterministically and record policy reason
Tool implementation exception	Only if classified transient	Sanitize error; never leak stack secrets

17.6 Multi-tool calls in a single turn

A model may request several tools in one assistant message—for example, two independent retrieval queries followed by compression. Each call has a unique identifier, and every executed or skipped call must receive a corresponding result message. The protocol is a ledger: request and result pairs must balance.

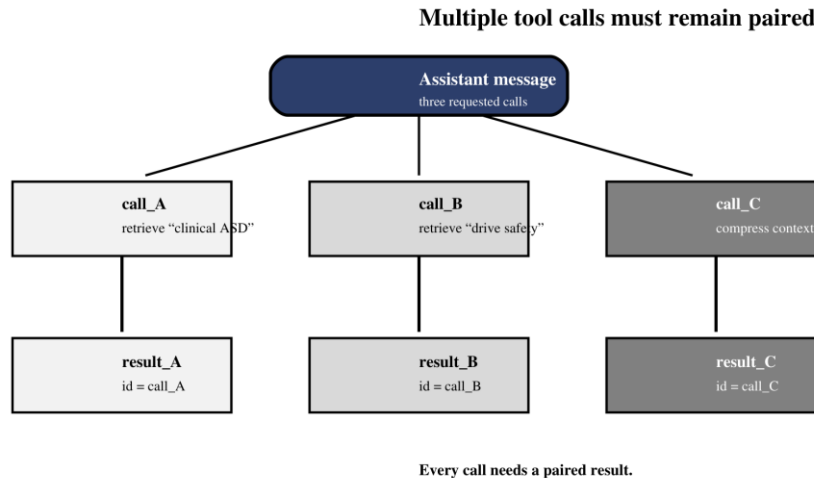


Figure 17.2 — Every requested call requires an explicitly paired result or skip record.

Figure 17.2 makes the accounting rule visible: every call ID reaches one result record, including calls rejected by policy. This one-to-one pairing keeps provider protocols valid and lets an audit distinguish a skipped operation from a lost message.

Parallel execution is safe only when calls are independent and their results can be merged deterministically. Two read-only retrievals may run concurrently. Compression cannot run before those retrievals have completed because it depends on their accumulated state. Preserve the model’s requested identifiers even if execution order changes, and sort or reduce results through explicit application logic.

```
for call in assistant.tool_calls:
    try:
        args = validate(call.name, call.arguments)
        content = dispatch(call.name, args)
    except ToolRejected as exc:
        content = f"SKIPPED: {exc}"

    messages.append({
        "role": "tool",
        "tool_call_id": call.id,
        "content": content,
    })
```

Common pitfall — Deleting tool-result messages

Memora learned to replace large raw retrieval contents with a short placeholder after compression, rather than deleting the messages. Deletion breaks the assistant-to-`tool_call_id` pairing required by

chat APIs; content scrubbing preserves the protocol while reducing context.

17.7 Safety: validating and sandboxing tool inputs

Tool safety begins before the function runs and continues after it returns. Validate the name against an allow-list, parse arguments with a typed schema, normalize strings, enforce authorization, restrict paths and network destinations, limit result size, apply timeouts, and remove secrets from logs. A tool should expose one bounded capability—not a general-purpose escape hatch.

Boundary	Control	Example
Selection	Allow-list tool names by phase	Retrieval phase cannot invoke deployment tools
Shape	Typed schema and unknown-field rejection	Only a non-empty `query` string is accepted
Authority	Derive permissions from request context	Model cannot claim a different user ID
Resources	Timeout, rate, and result-size limits	Return top bounded chunks, not an entire collection
Data	Path and destination allow-lists	Reject traversal outside an approved corpus root
Output	Sanitize and label untrusted content	Documents remain evidence, never system instructions

Analogy — A capability key

A safe tool is like a key cut for one door. It may open the approved retrieval operation, but it cannot unlock the operating system, database administration, or arbitrary network access. The narrower the key, the easier it is to reason about misuse.

17.7.1 Testing the boundary adversarially

A happy-path unit test proves only that valid input works. Boundary tests should also submit an extra property, an empty query, an oversized query, an unknown tool name, a valid tool in the wrong phase, a mismatched call ID, and a result large enough to exceed the context budget. Add a document whose text asks the agent to call another tool or reveal credentials. The expected outcome is not that the model always ignores the attack; it is that deterministic policy makes the prohibited transition impossible.

Then test observability. Each attempt should record the run ID, call ID, selected tool, validation outcome, duration, bounded result size, and a sanitized error category. Do not log credentials, full private documents, or unrestricted arguments merely because debugging is convenient. A production tool layer is successful when operators can reconstruct what happened without creating a second sensitive-data store in the logs.

17.7.2 A practical tool acceptance checklist

Before registering a tool, verify that its purpose can be stated as one bounded verb, its public arguments contain no server secrets or policy knobs, and its result has a documented size limit. Confirm that authorization uses authenticated context rather than model claims, that timeouts and retries have a shared run budget, and that errors are safe to show to the model. Finally, prove there is a test for an unknown field, forbidden phase, unauthorized identity, oversized result, and implementation failure.

This checklist often reveals that one proposed tool is really several capabilities. A `'manage_documents'` function that can search, upload, delete, and export should become separate operations with separate permissions. Narrow tools produce clearer schemas, smaller blast radii, and more useful audit records. They also make the agent's behavior easier to evaluate because each call expresses a specific intent.